Lecture 16

Fundamentals of Numerical Analysis

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

Next Topics

1. Interpolation: Approximating a function $f(x)$ by a polynomial $p_n(x)$.
2. Differentiation: Approximating the derivative of a function $f(x)$.
3. Integration: Approximating an integral $\int_a^b f(x)\,dx$

Summary so far:

- Monomials: $p(x) = a_0 + a_1 x + \cdots + a_n x^n$ results in poor conditioning
- Monomials: but evaluating the Monomial interpolant is cheap (nested iteration)
- Lagrange: $p(x) = \ell_0(x) y_0 + \cdots + \ell_n(x) y_n$ is very well behaved.
- Lagrange: but evaluating the Lagrange interpolant is expensive (each basis function is of the same order and the interpolant is not easily reduced to nested form)

Recall: Monomials

Obvious attempt: try picking

$$p(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

So for each x_i we have

$$p(x_i) = a_0 + a_1 x_i + a_2 x_i^2 + \cdots + a_n x_i^n = y_i$$

OR

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ & & & \vdots & \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}$$

That is,

$$a = M^{-1} y$$

Very bad matrix: terribly ill-conditioned, inverse entries are **large**
Very bad evaluation: values are **huge**

Lagrange

The general Lagrange form is

$$\ell_k(x) = \prod_{i=0, i\neq k}^{n} \frac{x - x_i}{x_k - x_i}$$

The resulting interpolating polynomial is

$$p(x) = \sum_{k=0}^{n} \ell_k(x) y_k$$

Example

Find the equation of the parabola passing through the points (1,6), (-1,0), and (2,12)

$x_0 = 1, x_1 = -1, x_2 = 2; \qquad y_0 = 6, y_1 = 0, y_2 = 12;$

$$\begin{aligned}
\ell_0(x) &= \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)} = \frac{(x+1)(x-2)}{(2)(-1)} \\
\ell_1(x) &= \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)} = \frac{(x-1)(x-2)}{(-2)(-3)} \\
\ell_2(x) &= \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)} = \frac{(x-1)(x+1)}{(1)(3)}
\end{aligned}$$

$$\begin{aligned}
p_2(x) &= y_0\ell_0(x) + y_1\ell_1(x) + y_2\ell_2(x) \\
&= -3 \times (x+1)(x-2) + 0 \times \frac{1}{6}(x-1)(x-2) \\
&\quad +4 \times (x-1)(x+1) \\
&= (x+1)[4(x-1) - 3(x-2)] \\
&= (x+1)(x+2)
\end{aligned}$$

Fixing Monomials, Fixing Lagrange

Back to the gas price example. Suppose we use a better basis like

$$(x - \bar{x})^k$$

instead of

$$x^k$$

For example, $\bar{x} = average(x_i), i = 0, \ldots, n$.
The basis $(x - \bar{x})^k$ are called *shifted monomials* because x is shifted by $\bar{x}$.

Recall Nested Form

Given a polynomial

$$p(x) = -5 + 4x - 7x^2 + 2x^3 + 3x^4$$

we can write this as

$$p(x) = -5 + x(4 + x(-7 + x(2 + 3x)));$$

evaluation can be done from the inside-out, for cheap (nested evaluation). This polynomial can also be written as

$$p(x) = -5 + 2x - 4x(x-1) + 8x(x-1)(x+1) + 3x(x-1)(x+1)(x-2)$$

in nested form

$$p(x) = -5 + x(2 + (x-1)(-4 + (x+1)(8 + 3(x-2))))$$

Newton Polynomials

- Newton Polynomials are of the form

$$p_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + a_3(x - x_0)(x - x_1)(x - x_2) + \ldots$$

- The basis used is thus

function	order
1	0
$x - x_0$	1
$(x - x_0)(x - x_1)$	2
$(x - x_0)(x - x_1)(x - x_2)$	3

- More stable that monomials
- More computationally efficient (nested iteration) than using Lagrange and shifted monomials

Newton Polynomials using Divided Differences

Consider the data

x_0	x_1	x_2
y_0	y_1	y_2

We want to find a_0, a_1, and a_2 in the following polynomial so that it fits the data:

$$p_2(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1)$$

Matching the data gives three equations to determine our three unknowns a_i:

$$\begin{aligned}
&\text{at } x_0\text{: } y_0 = a_0 + 0 + 0 \\
&\text{at } x_1\text{: } y_1 = a_0 + a_1(x_1 - x_0) + 0 \\
&\text{at } x_2\text{: } y_2 = a_0 + a_1(x_2 - x_0) + a_2(x_2 - x_0)(x_2 - x_1)
\end{aligned}$$

Newton Polynomials using Divided Differences

Or in matrix form:

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & x_1 - x_0 & 0 \\ 1 & x_2 - x_0 & (x_2 - x_0)(x_2 - x_1) \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} \begin{bmatrix} y_0 \\ y_1 \\ y_2 \end{bmatrix}$$

$\Rightarrow$ lower triangular
$\Rightarrow$ only $\mathcal{O}(n^2)$ operations

Question

How many operations are needed to find the coefficients in the monomial basis?

Newton Polynomials using Divided Differences

Using Forward Substitution to solve this lower triangular system yields:

$$
\begin{aligned}
a_0 &= y_0 = f(x_0) \\
a_1 &= \frac{y_1 - a_0}{x_1 - x_0} \\
&= \frac{f(x_1) - f(x_0)}{x_1 - x_0} \\
a_2 &= \frac{y_2 - a_0 - (x_2 - x_0)a_1}{(x_2 - x_1)(x_2 - x_0)} \\
&= \frac{f(x_2) - f(x_0) - (x_2 - x_0)\frac{f(x_1) - f(x_0)}{x_1 - x_0}}{(x_2 - x_1)(x_2 - x_0)} \\
&= \frac{\frac{f(x_2) - f(x_1)}{x_2 - x_1} - \frac{f(x_1) - f(x_0)}{x_1 - x_0}}{x_2 - x_0}
\end{aligned}
$$

Newton Polynomials using Divided Differences

From this we see a pattern. There are many terms of the form

$$\frac{f(x_j) - f(x_i)}{x_j - x_i}$$

These are called *divided differences* and are denoted with square brackets:

$$f[x_i, x_j] = \frac{f(x_j) - f(x_i)}{x_j - x_i}$$

Applying this to our results:

$$\begin{aligned} a_0 &= f[x_0] \\ a_1 &= f[x_0, x_1] \\ a_2 &= \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} \\ &= f[x_0, x_1, x_2] \end{aligned}$$

Newton Polynomials using Divided Differences

example: long way

Example

For the data

x	1	-4	0
y	3	13	-23

Find the 2nd order interpolatin polynomial using Newton.

We know

$$p_1(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1)$$

And that

$$\begin{aligned}
a_0 &= f[x_0] = f[1] = f(1) = 3 \\
a_1 &= f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0} = \frac{13 - 3}{-4 - 1} = -2 \\
a_2 &= f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} \\
&= \frac{\frac{-23-13}{0--4} - \frac{13-3}{-4-1}}{0 - 1} \\
&= \frac{-9 + 2}{-1} = 7
\end{aligned}$$

So

$$p_1(x) = 3 - 2(x - 1) + 7(x - 1)(x + 4)$$

Divided Differences

Recursive Property

$$f[x_0, \ldots, x_k] = \frac{f[x_1, \ldots, x_k] - f[x_0, \ldots, x_{k-1}]}{x_k - x_0}$$

With the first two defined by

$$f[x_i] = f(x_i)$$

$$f[x_i, x_j] = \frac{f[x_j] - f[x_i]}{x_j - x_i}$$

Divided Differences

Invariance Theorem

$f[x_0, \ldots, x_k]$ is invariant under all permutations of the arguments $x_0, \ldots, x_k$

Simple "proof": $f[x_0, x_1, ..., x_k]$ is the coefficient of the x^k term in the polynomial interpolating f at $x_0, \ldots, x_k$. But any permutation of the x_i still gives the same polynomial.
This says that we can also write

$$f[x_i, \ldots, x_j] = \frac{f[x_{i+1}, \ldots, x_j] - f[x_i, \ldots, x_{j-1}]}{x_j - x_i}$$

Divided Differences

the easy way: tables

We can compute the divided differences much easier using tables. To construct the divided difference table for $f(x)$ for the $x_0, \ldots, x_3$

x	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
x_0	$f[x_0]$			
		$f[x_0,x_1]$		
x_1	$f[x_1]$		$f[x_0,x_1,x_2]$	
		$f[x_1,x_2]$		$f[x_0,x_1,x_2,x_3]$
x_2	$f[x_2]$		$f[x_1,x_2,x_3]$	
		$f[x_2,x_3]$		
x_3	$f[x_3]$			

Divided Differences

the easy way: tables

We can compute the divided differences much easier using tables. To construct the divided difference table for $f(x)$ for the $x_0, \ldots, x_3$

x	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
x_0	$f[x_0]$			
		$f[x_0,x_1]$		
x_1	$f[x_1]$		$f[x_0,x_1,x_2]$	
		$f[x_1,x_2]$		$f[x_0,x_1,x_2,x_3]$
x_2	$f[x_2]$		$f[x_1,x_2,x_3]$	
		$f[x_2,x_3]$		
x_3	$f[x_3]$			

Divided Differences

the easy way: tables

We can compute the divided differences much easier using tables. To construct the divided difference table for $f(x)$ for the $x_0, \ldots, x_3$

x	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
x_0	$f[x_0]$			
		$f[x_0,x_1]$		
x_1	$f[x_1]$		$f[x_0,x_1,x_2]$	
		$f[x_1,x_2]$		$f[x_0,x_1,x_2,x_3]$
x_2	$f[x_2]$		$f[x_1,x_2,x_3]$	
		$f[x_2,x_3]$		
x_3	$f[x_3]$			

Divided Differences

the easy way: tables

We can compute the divided differences much easier using tables. To construct the divided difference table for $f(x)$ for the $x_0, \ldots, x_3$

x	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
x_0	$f[x_0]$			
		$f[x_0,x_1]$		
x_1	$f[x_1]$		$f[x_0,x_1,x_2]$	
		$f[x_1,x_2]$		$f[x_0,x_1,x_2,x_3]$
x_2	$f[x_2]$		$f[x_1,x_2,x_3]$	
		$f[x_2,x_3]$		
x_3	$f[x_3]$			

Divided Differences

the easy way: tables

We can compute the divided differences much easier using tables. To construct the divided difference table for $f(x)$ for the $x_0, \ldots, x_3$

x	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
x_0	$f[x_0]$			
		$f[x_0,x_1]$		
x_1	$f[x_1]$		$f[x_0,x_1,x_2]$	
		$f[x_1,x_2]$		$f[x_0,x_1,x_2,x_3]$
x_2	$f[x_2]$		$f[x_1,x_2,x_3]$	
		$f[x_2,x_3]$		
x_3	$f[x_3]$			

Divided Differences

the easy way: tables

We can compute the divided differences much easier using tables. To construct the divided difference table for $f(x)$ for the $x_0, \ldots, x_3$

x	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
x_0	$f[x_0]$			
		$f[x_0,x_1]$		
x_1	$f[x_1]$		$f[x_0,x_1,x_2]$	
		$f[x_1,x_2]$		$f[x_0,x_1,x_2,x_3]$
x_2	$f[x_2]$		$f[x_1,x_2,x_3]$	
		$f[x_2,x_3]$		
x_3	$f[x_3]$			

Divided Differences

the easy way: tables

We can compute the divided differences much easier using tables. To construct the divided difference table for $f(x)$ for the $x_0, \ldots, x_3$

x	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
x_0	$f[x_0]$			
		$f[x_0,x_1]$		
x_1	$f[x_1]$		$f[x_0,x_1,x_2]$	
		$f[x_1,x_2]$		$f[x_0,x_1,x_2,x_3]$
x_2	$f[x_2]$		$f[x_1,x_2,x_3]$	
		$f[x_2,x_3]$		
x_3	$f[x_3]$			

Divided Differences

the easy way: example

Construct the divided differences table for the data

x	1	$\frac{3}{2}$	0	2
y	3	$\frac{13}{4}$	3	$\frac{5}{3}$

and construct the largest order interpolating polynomial.
We can compute the divided differences much more easily using tables. To construct the divided difference table for $f(x)$ for the $x_0, \ldots, x_3$

x	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
1	3			
		$\frac{1}{2}$		
$\frac{3}{2}$	$\frac{13}{4}$		$\frac{1}{3}$	
		$\frac{1}{6}$		-2
0	3		$-\frac{5}{3}$	
		$-\frac{2}{3}$		
2	$\frac{5}{3}$			

Divided Differences

the easy way: example

Construct the divided differences table for the data

x	1	$\frac{3}{2}$	0	2
y	3	$\frac{13}{4}$	3	$\frac{5}{3}$

and construct the largest order interpolating polynomial.
We can compute the divided differences much more easily using tables. To construct the divided difference table for $f(x)$ for the $x_0, \ldots, x_3$

x	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
1	3			
		$\frac{1}{2}$		
$\frac{3}{2}$	$\frac{13}{4}$		$\frac{1}{3}$	
		$\frac{1}{6}$		-2
0	3		$-\frac{5}{3}$	
		$-\frac{2}{3}$		
2	$\frac{5}{3}$			

Divided Differences

the easy way: example

Construct the divided differences table for the data

x	1	$\frac{3}{2}$	0	2
y	3	$\frac{13}{4}$	3	$\frac{5}{3}$

and construct the largest order interpolating polynomial.
We can compute the divided differences much more easily using tables. To construct the divided difference table for $f(x)$ for the $x_0, \ldots, x_3$

x	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
1	3			
		$\frac{1}{2}$		
$\frac{3}{2}$	$\frac{13}{4}$		$\frac{1}{3}$	
		$\frac{1}{6}$		-2
0	3		$-\frac{5}{3}$	
		$-\frac{2}{3}$		
2	$\frac{5}{3}$			

Divided Differences

the easy way: example

Construct the divided differences table for the data

x	1	$\frac{3}{2}$	0	2
y	3	$\frac{13}{4}$	3	$\frac{5}{3}$

and construct the largest order interpolating polynomial.
We can compute the divided differences much more easily using tables. To construct the divided difference table for $f(x)$ for the $x_0, \ldots, x_3$

x	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
1	3			
		$\frac{1}{2}$		
$\frac{3}{2}$	$\frac{13}{4}$		$\frac{1}{3}$	
		$\frac{1}{6}$		-2
0	3		$-\frac{5}{3}$	
		$-\frac{2}{3}$		
2	$\frac{5}{3}$			

Divided Differences

the easy way: example

Construct the divided differences table for the data

x	1	$\frac{3}{2}$	0	2
y	3	$\frac{13}{4}$	3	$\frac{5}{3}$

and construct the largest order interpolating polynomial.
We can compute the divided differences much more easily using tables. To construct the divided difference table for $f(x)$ for the $x_0, \ldots, x_3$

x	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
1	3			
		$\frac{1}{2}$		
$\frac{3}{2}$	$\frac{13}{4}$		$\frac{1}{3}$	
		$\frac{1}{6}$		-2
0	3		$-\frac{5}{3}$	
		$-\frac{2}{3}$		
2	$\frac{5}{3}$			

Divided Differences

the easy way: example

Construct the divided differences table for the data

x	1	$\frac{3}{2}$	0	2
y	3	$\frac{13}{4}$	3	$\frac{5}{3}$

and construct the largest order interpolating polynomial.
We can compute the divided differences much more easily using tables. To construct the divided difference table for $f(x)$ for the $x_0, \ldots, x_3$

x	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
1	3			
		$\frac{1}{2}$		
$\frac{3}{2}$	$\frac{13}{4}$		$\frac{1}{3}$	
		$\frac{1}{6}$		-2
0	3		$-\frac{5}{3}$	
		$-\frac{2}{3}$		
2	$\frac{5}{3}$			

Divided Differences

the easy way: example

Construct the divided differences table for the data

x	1	$\frac{3}{2}$	0	2
y	3	$\frac{13}{4}$	3	$\frac{5}{3}$

and construct the largest order interpolating polynomial.
We can compute the divided differences much more easily using tables. To construct the divided difference table for $f(x)$ for the $x_0, \ldots, x_3$

x	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
1	3			
		$\frac{1}{2}$		
$\frac{3}{2}$	$\frac{13}{4}$		$\frac{1}{3}$	
		$\frac{1}{6}$		-2
0	3		$-\frac{5}{3}$	
		$-\frac{2}{3}$		
2	$\frac{5}{3}$			

Divided Differences

the easy way: example

x	$f[\cdot]$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$
1	3			
		$\frac{1}{2}$		
$\frac{3}{2}$	$\frac{13}{4}$		$\frac{1}{3}$	
		$\frac{1}{6}$		-2
0	3		$-\frac{5}{3}$	
		$-\frac{2}{3}$		
2	$\frac{5}{3}$			

The coeffiecients are readily available and we arrive at

$$p_2(x) = 3 + \frac{1}{2}(x-1) + \frac{1}{3}(x-1)(x-\frac{3}{2}) - 2(x-1)(x-\frac{3}{2})x$$

Recall: Newton Polynomials

- Newton Polynomials are of the form

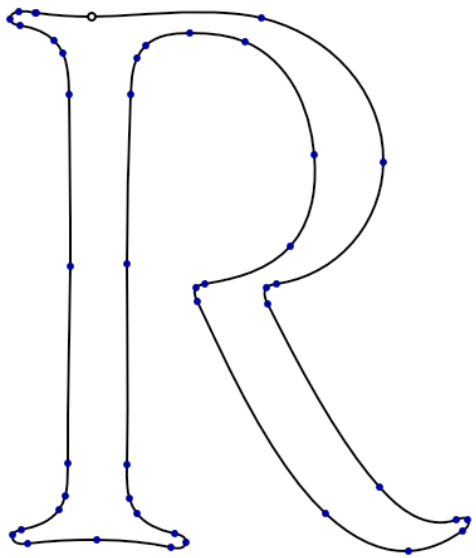
$$p_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + a_3(x - x_0)(x - x_1)(x - x_2) + \dots$$

- The basis used is thus

function	order
1	0
$x - x_0$	1
$(x - x_0)(x - x_1)$	2
$(x - x_0)(x - x_1)(x - x_2)$	3

- More stable evaluation than monomials
- More computationally efficient (nested iteration) than using Lagrange

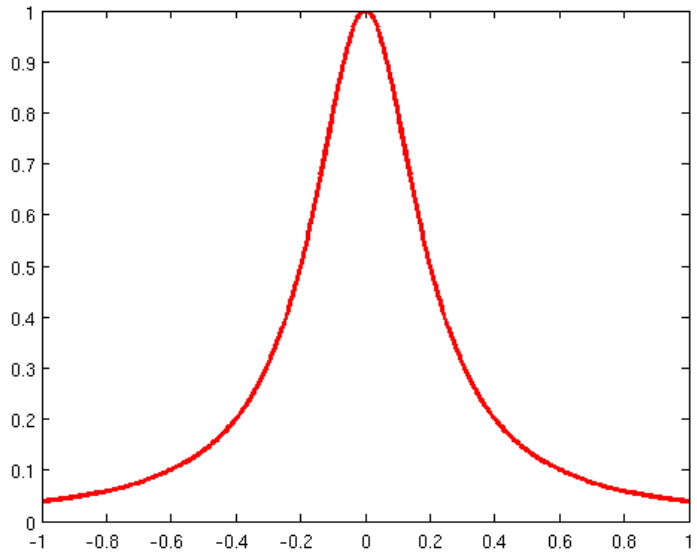
Fonts == interpolation



- how do we ”contain” our interpolation?
- splines
- Postscript (Adobe): rasterization on-the-fly. Fonts, etc are defined as cubic Bézier curves (linear interpolation between lower order Bézier curves)
- TrueType (Apple): similar, quadratic Bézier curves, thus cannot convert from TrueType to PS (Type1) losslessly

How bad is polynomial interpolation?

Let's take something very smooth function



How does interpolation behave?